

Curso SQL intermedio – Clase 4

Temas:

Consultas multitaslas: Alias de Tablas, NATURAL JOINS, USING, INNER JOINS, OUTER JOINS, RIGHT/LEFT/FULL JOINS.

Consultas multitaslas

Para obtener resultados de consultas de varias tablas se emplean diversas técnicas en SQL.

Obtención de Datos de Varias Tablas

EMPLOYEES

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
100	King	90
101	Kochhar	90
...		
202	Fay	20
205	Higgins	110
206	Gietz	110

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
50	Shipping	1500
60	IT	1400
80	Sales	2500
90	Executive	1700
110	Accounting	1700
190	Contracting	1700



EMPLOYEE_ID	DEPARTMENT_ID	DEPARTMENT_NAME
200	10	Administration
201	20	Marketing
202	20	Marketing
...		
102	90	Executive
205	110	Accounting
206	110	Accounting

Una de estas técnicas es la reunión (JOIN). Las reuniones compatibles con el estándar SQL son las siguientes:

- Reuniones cruzadas.
- Reuniones naturales.
- Cláusula USING.
- Reuniones externas completas (o de dos lados).
- Condiciones de reunión arbitrarias para reuniones externas.

Definición de reuniones según estándar SQL

```
SELECT    table1.column, table2.column  
FROM      table1  
[NATURAL JOIN table2] |  
[JOIN table2 USING (column_name)] |  
[JOIN table2  
    ON (table1.column_name = table2.column_name)] |  
[LEFT|RIGHT|FULL OUTER JOIN table2  
    ON (table1.column_name = table2.column_name)] |  
[CROSS JOIN table2];
```

En el cuadro:

table1.column muestra la tabla y la columna de las que se recuperan los datos.

NATURAL JOIN reúne dos tablas basándose en el mismo nombre de columna.

JOIN table USING column_name realiza una reunión equitativa o por igualdad basándose en el nombre de columna.

JOIN table ON table1.column_name realiza una reunión por igualdad basándose en la condición de la cláusula ON, = table2.column_name

LEFT/RIGHT/FULL OUTER se utiliza para realizar reuniones externas.

CROSS JOIN devuelve un producto cartesiano de las dos tablas.

Creación de reuniones naturales

La cláusula NATURAL JOIN se realiza a partir de todas las columnas de las dos tablas que tienen el mismo nombre.

Selecciona filas de las dos tablas que tengan valores idénticos en todas las columnas correspondientes.

Si las columnas que tengan los mismos nombres, tienen tipos de datos diferentes, se devuelve un error.

Ejemplo:

En el ejemplo de abajo, la tabla LOCATIONS se reúne con la tabla DEPARTMENT por la columna LOCATION_ID, que es la única con el mismo nombre en ambas tablas. De haber habido más columnas comunes, la reunión las habría utilizado a todas.

```
SELECT department_id, department_name,
       location_id, city
FROM   departments
NATURAL JOIN locations ;
```

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID	CITY
60	IT	1400	Southlake
50	Shipping	1500	South San Francisco
10	Administration	1700	Seattle
90	Executive	1700	Seattle
110	Accounting	1700	Seattle
190	Contracting	1700	Seattle
20	Marketing	1800	Toronto
80	Sales	2500	Oxford

8 rows selected.

Reuniones naturales con una cláusula WHERE

Las restricciones adicionales de una reunión natural se implementan mediante la cláusula WHERE. El ejemplo siguiente limita las filas de salida a aquellas con un identificador de departamento igual a 20 ó 50.

```
SELECT department_id, department_name, location_id, city
FROM   hr.departments
NATURAL JOIN locations
WHERE  department_id IN (20, 50);
```

Creación de reuniones con la cláusula USING

Si hay varias columnas que tienen los mismos nombres pero los tipos de datos no se corresponden, la cláusula NATURAL JOIN se puede modificar mediante la cláusula USING para especificar las columnas que se deben utilizar para una reunión por igualdad.

- Utilice la cláusula USING para asignar sólo una columna cuando corresponde a más de una columna.
- No utilice un alias o un nombre de tabla en las columnas a las que se hace referencia.
- Las cláusulas NATURAL JOIN y USING se excluyen mutuamente.

Cláusula USING

Las uniones naturales pueden utilizar todas las columnas con nombres y tipos de datos correspondientes para unir las tablas. Se puede utilizar la cláusula USING para especificar únicamente las columnas que se deben utilizar para una unión igualitaria. Las columnas a las que se hace referencia en la cláusula USING no deben tener cualificador (nombre de tabla o alias) en ninguna parte de la sentencia SQL.

Por ejemplo, esta sentencia es válida:

```
SELECT l.city, d.department_name
FROM hr.locations l JOIN hr.departments d USING (location_id)
WHERE location_id = 1400;
```

La sentencia siguiente no es válida porque la cláusula WHERE cualifica a LOCATION_ID:

```
SELECT l.city, d.department_name
FROM hr.locations l JOIN hr.departments d USING (location_id)
WHERE d.location_id = 1400;
```

ORA-25154: column part of USING clause cannot have qualifier

Se aplica la misma restricción a las reuniones naturales. Por tanto, las columnas con el mismo nombre en ambas tablas se deben utilizar sin cualificadores.

Reunión por nombres de columnas

EMPLOYEES

EMPLOYEE_ID	DEPARTMENT_ID
200	10
201	20
202	20
124	50
141	50
142	50
143	50
144	50
103	60
104	60
107	60
149	80
174	80
176	80

...

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
20	Marketing
50	Shipping
50	Shipping
50	Shipping
50	Shipping
50	Shipping
50	Shipping
60	IT
60	IT
60	IT
80	Sales
80	Sales
80	Sales



...



Clave ajena

Clave primaria

La cláusula USING para reuniones por igualdad

Para determinar el nombre de departamento de un empleado, compare el valor de la columna DEPARTMENT_ID de la tabla HR.EMPLOYEES con los valores de DEPARTMENT_ID de la tabla HR.DEPARTMENTS. La relación entre las tablas HR.EMPLOYEES y HR.DEPARTMENTS es una reunión por igualdad (es decir, los valores de la columna DEPARTMENT_ID de ambas tablas deben ser los mismos). Con frecuencia, este tipo de reunión implica complementos de clave primaria y foránea.

Nota: Las reuniones por igualdad se denominan también reuniones simples o internas.

```
SELECT employees.employee_id, employees.last_name,  
       departments.location_id, department_id  
FROM   employees JOIN departments  
USING (department_id);
```

EMPLOYEE_ID	LAST_NAME	LOCATION_ID	DEPARTMENT_ID
200	Whalen	1700	10
201	Hartstein	1800	20
202	Fay	1800	20
124	Mourgos	1500	50
141	Rajs	1500	50
142	Davies	1500	50
144	Vargas	1500	50
143	Matos	1500	50

...

19 rows selected.

El ejemplo reúne la columna DEPARTMENT_ID de las tablas EMPLOYEES y DEPARTMENTS, con lo que muestra la ubicación en la que trabaja un empleado.

Cualificación de Nombres de Columna Ambiguos

- Utilice prefijos de tabla para cualificar nombres de columna que estén en varias tablas.
- Utilice prefijos de tabla para mejorar el rendimiento.
- Utilice alias de columna para distinguir columnas que tengan nombres idénticos pero que residan en tablas diferentes.
- No utilice alias en columnas que estén identificadas en la cláusula USING y que se muestren en cualquier otra parte de la sentencia SQL.

Debe cualificar los nombres de las columnas con el nombre de la tabla para evitar ambigüedades. Sin los prefijos de tabla, la columna DEPARTMENT_ID en la lista SELECT podría ser de la tabla HR.DEPARTMENTS o de la tabla HR.EMPLOYEES. Resulta necesario agregar el prefijo de tabla para ejecutar la consulta:

```
SELECT employees.employee_id, employees.last_name, departments.department_id,  
       departments.location_id
```

```
FROM   hr.employees JOIN hr.departments
```

```
ON     hr.employees.department_id = hr.departments.department_id;
```

Si no existen nombres de columna comunes en las dos tablas, no es necesario cualificar las columnas. Sin embargo, utilizar el prefijo de tabla mejora el rendimiento, ya que le indica a Oracle Server exactamente dónde puede encontrar las columnas.

Nota: Al realizar uniones mediante la cláusula USING, no puede cualificar una columna que se utilice en la propia cláusula USING. Lo que es más, si se utiliza esa columna en cualquier otra parte de la sentencia SQL, no puede aplicarle un alias.

Uso de Alias de Tabla

Cualificar nombres de columna con nombres de tabla puede llevar mucho tiempo, especialmente si los nombres de tabla son largos. Puede utilizar alias de tabla en lugar de nombres de tabla. Al igual que los alias de columna cambian el nombre de las columnas, los alias de tabla hacen lo propio con las tablas. Los alias de tabla ayudan a reducir el código SQL, con lo que se consume menos memoria. Ejemplo:

```
SELECT e.employee_id, e.last_name, d.location_id, department_id
FROM   hr.employees e JOIN hr.departments d
USING (department_id);
```

Observe cómo se identifican en el ejemplo los alias de tabla de la cláusula FROM. El nombre de tabla se especifica entero, seguido de un espacio y, a continuación, el alias de tabla. Se le ha dado a la tabla EMPLOYEES el alias e y a la tabla DEPARTMENTS, el alias d.

Resumen uso de alias de tablas

- Los alias de tabla pueden contener hasta 30 caracteres, pero son mejores los cortos que los largos.
- Si se utiliza un alias de tabla para un nombre de tabla concreto en la cláusula FROM, se debe sustituir el nombre de tabla por alias de tabla en toda la sentencia SELECT.
- Los alias de tabla deben ser significativos.
- Los alias de tabla son válidos únicamente para la sentencia SELECT actual.
- Utilice alias de tabla para simplificar las consultas.
- Utilice alias de tabla para mejorar el rendimiento.

Creación de reuniones con la cláusula ON

- La condición de reunión para la reunión natural es básicamente una reunión por igualdad de todas las columnas con el mismo nombre.
- Utilice la cláusula ON para especificar condiciones arbitrarias o para especificar las columnas que se reunirán.
- La condición de reunión se separa de otras condiciones de búsqueda.
- La cláusula ON facilita la comprensión del código.

Utilice la cláusula ON para especificar una condición de reunión. Esto le permite especificar condiciones de reunión aparte de cualquier condición de búsqueda o de filtro en la cláusula WHERE.

```

SELECT e.employee_id, e.last_name, e.department_id,
       d.department_id, d.location_id
FROM   employees e JOIN departments d
ON     (e.department_id = d.department_id);

```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_ID	LOCATION_ID
200	Whalen	10	10	1700
201	Hartstein	20	20	1800
202	Fay	20	20	1800
124	Mourgos	50	50	1500
141	Rajs	50	50	1500
142	Davies	50	50	1500
143	Matos	50	50	1500

...

19 rows selected.

En este ejemplo, las columnas DEPARTMENT_ID de las tablas HR.EMPLOYEES y HR.DEPARTMENTS se reúnen mediante la cláusula ON. Siempre que un identificador de departamento de la tabla HR.EMPLOYEES sea igual a un identificador de departamento de la tabla DEPARTMENTS, se devuelve la fila.

También puede utilizar la cláusula ON para reunir columnas con nombres diferentes.

Autoreuniones mediante la cláusula ON

EMPLOYEES (WORKER)

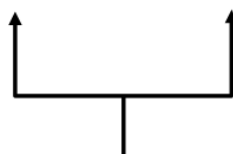
EMPLOYEE_ID	LAST_NAME	MANAGER_ID
100	King	
101	Kochhar	100
102	De Haan	100
103	Hunold	102
104	Ernst	103
107	Lorentz	103
124	Mourgos	100

...

EMPLOYEES (MANAGER)

EMPLOYEE_ID	LAST_NAME
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst
107	Lorentz
124	Mourgos

...



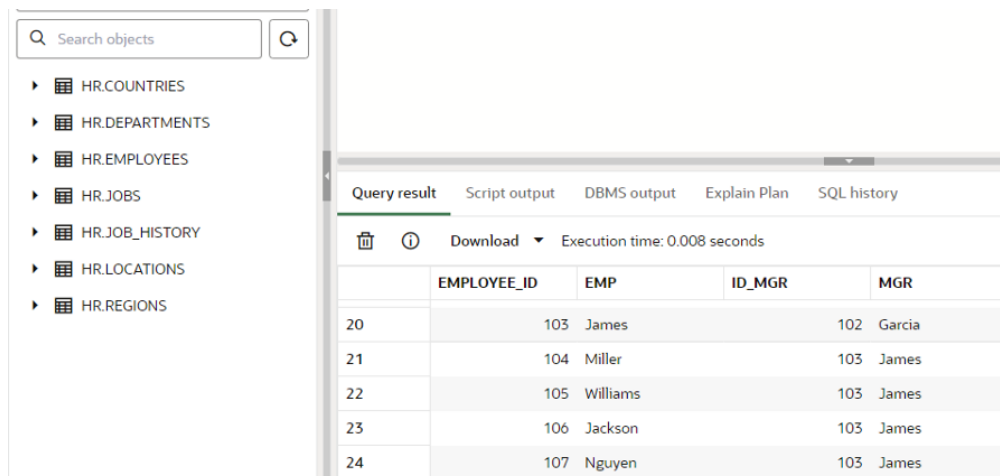
**MANAGER_ID en la tabla WORKER es igual que
EMPLOYEE_ID en la tabla MANAGER.**

Reunión de una tabla consigo misma

A veces, tendrá que combinar una tabla consigo misma. Para encontrar el nombre del supervisor de cada empleado, debe reunir la tabla HR.EMPLOYEES consigo misma o realizar una autoreunión. Por ejemplo, para encontrar el nombre del supervisor de Miller, debe:

- Buscar Miller en la tabla HR.EMPLOYEES en la columna LAST_NAME.
- Buscar el número de supervisor para Miller en la columna MANAGER_ID. El número del supervisor de Miller es 103.
- Buscar el nombre del supervisor con el valor EMPLOYEE_ID 103 en la columna LAST_NAME. El número de empleado de James es 103, por lo que James es el supervisor de Miller.

En este proceso, se consulta la tabla dos veces. La primera vez, se consulta la tabla es para buscar Miller en la columna LAST_NAME y el valor MANAGER_ID de 103. La segunda vez, se consulta la columna EMPLOYEE_ID para buscar 103 y la columna LAST_NAME para buscar James.



	EMPLOYEE_ID	EMP	ID_MGR	MGR
20	103	James	102	Garcia
21	104	Miller	103	James
22	105	Williams	103	James
23	106	Jackson	103	James
24	107	Nguyen	103	James

La cláusula ON también se puede utilizar para reunir columnas con nombres diferentes, dentro de la misma tabla o en una tabla diferente.

El ejemplo que se muestra en una autoreunión de la tabla HR.EMPLOYEES, basada en las columnas EMPLOYEE_ID y MANAGER_ID.

Aplicación de Condiciones Adicionales a una Unión

Puede aplicar condiciones adicionales a la unión. Ejemplo:

```
SELECT e.employee_id, e.last_name, e.department_id Emp_dept_id, d.department_id,  
d.location_id  
FROM hr.employees e JOIN hr.departments d  
ON (e.department_id = d.department_id)  
WHERE e.manager_id = 149;
```


Query result Script output DBMS output Explain Plan SQL history					
Download Execution time: 0.001 seconds					
	EMPLOYEE_ID	LAST_NAME	EMP_DEPT_ID	DEPARTMENT_ID	LOCATION_ID
1	174	Abel	80	80	2500
2	175	Hutton	80	80	2500
3	179	Johnson	80	80	2500
4	177	Livingston	80	80	2500
5	176	Taylor	80	80	2500

El ejemplo realiza una reunión de las tablas HR.EMPLOYEES y HR.DEPARTMENTS, y además muestra sólo a los empleados con el identificador de supervisor 149. Para agregar condiciones adicionales a la cláusula ON, puede agregar cláusulas AND. De forma alternativa, puede utilizar una cláusula WHERE para aplicar condiciones adicionales.

Reuniones multitabla

```
SELECT employee_id, city, department_name
FROM   employees e
JOIN   departments d
ON     d.department_id = e.department_id
JOIN   locations l
ON     d.location_id = l.location_id;
```

EMPLOYEE_ID	CITY	DEPARTMENT_NAME
103	Southlake	IT
104	Southlake	IT
107	Southlake	IT
124	South San Francisco	Shipping
141	South San Francisco	Shipping
142	South San Francisco	Shipping
143	South San Francisco	Shipping
144	South San Francisco	Shipping

...
19 rows selected.

Una reunión multitabla es una unión de tres o más tablas. En la sintaxis compatible con SQL:1999, las reuniones se realizan de izquierda a derecha. De esta forma, la primera reunión que se debe realizar es HR.EMPLOYEES JOIN HR.DEPARTMENTS. La primera condición de reunión puede hacer referencia a columnas de HR.EMPLOYEES y HR.DEPARTMENTS pero no puede hacer referencia a columnas de HR.LOCATIONS. La segunda condición de reunión puede hacer referencia a columnas de las tres tablas.

Reuniones no equitativas

Una reunión no equitativa es aquella que contiene algo distinto a un operador de igualdad. La relación entre las tablas HR.EMPLOYEES y HR.JOB_HISTORY es un ejemplo de reunión no equitativa. Una relación entre las dos tablas es que la columna SALARY de la tabla HR.EMPLOYEES debe estar entre los valores de las columnas LOWEST_SALARY y HIGHEST_SALARY de la tabla HR.JOB_HISTORY. La relación se obtiene mediante un operador distinto al de igualdad (=).

Ejemplo:

```
SELECT e.last_name, e.salary, j.job_title
FROM hr.employees e JOIN hr.jobs j
ON e.salary
    BETWEEN j.min_salary AND j.max_salary;
```

El ejemplo crea una reunión no equitativa para evaluar el grado del salario de un empleado. El salario debe estar entre cualquier par de los rangos de salario mínimo y máximo.

Nota: Se pueden utilizar otras condiciones (como <= y >=), pero BETWEEN es la más sencilla. Recuerde especificar el valor bajo en primer lugar y el alto al final cuando utilice BETWEEN.

Práctica

1. Muestre nombre de empleados y nombre de su departamento.
2. Muestre empleados y ciudad donde trabajan.
3. Muestre empleados y nombre del país donde trabajan.
4. Muestre empleados y nombre de su puesto (job_title).
5. Muestre empleados y región donde trabajan.
6. Muestre empleados que trabajan en ciudades de Canadá.
7. Desarrolle una consulta que liste el código, nombre y apellido de los empleados y sus respectivos jefes con título Empleado y Jefe.
8. Muestre empleados cuyo departamento tiene más de 5 empleados, y la ubicación del departamento.
9. Muestre empleados que trabajan en regiones llamadas 'Americas'.
10. Desarrolle una consulta que liste los países por región; los datos que debe mostrar son: el código de la región y nombre de la región con los nombres de sus países.
11. Realice una consulta que muestre el código, nombre, apellido, inicio y fin del historial de trabajo de los empleados.
12. Elabore una consulta que muestre el nombre y apellido del empleado con título Empleado, el salario, porcentaje de comisión, la comisión y salario total. Elabore una consulta que liste nombre del trabajo y el salario de los empleados que son jefes, cuyo código es 100 o 125 y cuyo salario sea mayor de 6000.
13. Realice una consulta que muestre el código, nombre de la región y el nombre de los países que se encuentran en “Asia”.

14. Desarrolle una consulta que liste el código de la localidad, la ciudad y el nombre del departamento de únicamente de los que se encuentran fuera de estados unidos (US).
15. Elabore una consulta que liste el código de la región y nombre de la región, código de la localidad, la ciudad, código del país y nombre del país, de solamente de las localidades mayores a 2400.
16. Desarrolle una consulta donde muestre el código de región con un alias de Región, el nombre de la región con una etiqueta Nombre Región, que muestre una cadena string (concatenación) que diga la siguiente frase “Código País: CA Nombre: Canadá”, CA es el código de país y Canadá es el nombre del país con etiqueta País, el código de localización con etiqueta Localización, la dirección de calle con etiqueta Dirección y el código postal con etiqueta “Código Postal”, esto a su vez no deben aparecer código postal que sean nulos.
17. Desarrolle una consulta que muestre el nombre de la región, el nombre del país, el estado de la provincia, el código de los empleados que son manager, el nombre y apellido del empleado que es manager de los países del Reino Unido (UK), Estados Unidos de América (US), respectivamente de los estados de Washington y Oxford.
18. Realice una consulta que muestre el nombre y apellido de los empleados que trabajan para departamentos que están localizados en países cuyo nombre comienza con la letra C, que muestre el nombre del país.